

Miércoles , 17 dic

REMINDER: Acknowledge the use of AI

En els treballs de l'estudiantat, l'ús literal de respostes generades per IA com si foren originals es pot considerar una acció fraudulenta, segons l'article 11 de la [normativa d'avaluació de grau i màster de l'UJI](#).

If you completed any work with the aid of an AI tool,
you should always acknowledge the use.

Para esta última entrega: Patrolling task: TD building in simulation and/or with the real robot, he utilizado la IA para una cosa en concreto, aunque en realidad era para hacer la mejora de mis versiones V4 y V5, según las especificaciones del profesor.

Básicamente era añadir el yaw en el código. Tal como dice Enric: "Com et deia en la versió anterior, en ROS quan enviem un "goal" amb RViz especifiquem tant la posició com l'orientació. Hauries d'extendre el teu codi per a incloure també l'orientació de manera que el robot detecta quan arriba a un goal si la distància és menor que un umbral i la diferència d'orientació també és menor que un altre umbral.

La diferència d'orientacions entre la pose actual i la goal es calcula convertint els quaternions a angles "yaw" i fent la diferència d'eixos angles."

Estuve intentándolo yo sola y no me acababa de salir, aunque al final descubrí que no era por un error de la funcionalidad del código sino por la exactitud de las coordenadas y de cómo las estaba anotando.

A continuación haré una breve explicación de mi código para la versión cinco en la que trabajamos con el tf2 para justificar que sí he entendido la ayuda que he tomado de la IA, en mi caso, de ChatGpt.

VERSION 5: TF2

Esto define el tipo de mensaje PoseStamped que contiene:

- o Una posición (x,y,z)
- o Una orientación (rotación)
- o Un frame-id (por ej. "map")
- o Un timestamp (hora)

Esto es lo que se usa para decirle al robot:
"Ve a esta posición y con esta orientación"

① // INCLUDES Y DEPENDENCIAS

Zela

```

1  #include <chrono> ~> manejo del tiempo, duraciones, relojes...
2  #include <cmath> ~> funciones matemáticas (hypot, fabs, M_PI, etc...). (raíces, valores absolutos,  $\pi$ , etc)
3  #include <vector> ~> contenedor dinámico para almacenar los waypoints. (lista de puntos)
4
5  #include "rclcpp/rclcpp.hpp" ~> biblioteca principal de ROS2 en C++ (SIN ESTO NO EXISTE ROS EN EL PROGRAMA)
6  #include "geometry_msgs/msg/pose_stamped.hpp" ~> Mensajes ROS para posiciones + orientación con frame y timestamp (PoseStamped)
7  #include "tf2_ros/transform_listener.h" ~> TF2: Permite escuchar transformaciones entre frames (map, base-link, etc)
8  #include "tf2_ros/buffer.h"
9  #include "tf2_geometry_msgs/tf2_geometry_msgs.hpp" ~> - Conversiones entre mensajes ROS y TF2 } Convertir orientaciones
10 #include "tf2/utils.h" ~> - Utilidades como getYaw() } sacar el "yaw" (giro) de un cuaternión
11 ② // NAMESPACE DE TIEMPO
12 using namespace std::chrono_literals; ~> Permite usar literales como 1s, 100ms, etc (aunque aquí no se usen explícitamente)
13 ③ // PARÁMETROS GLOBALES
14 // ===== PARÁMETROS globales
15 constexpr double DIST_TOL_CLOSE = 0.45; } Tolerancia de distancia para considerar que el robot llegó al waypoint.
16 constexpr double DIST_TOL_FAR = 0.30; } Se usan 2 valores dependiendo de si el robot está casi parado o aún en movimiento.
17 constexpr double YAW_TOL = 0.30; } Tolerancia de orientación (en radianes) = Distancia max para decir: "ya estoy en el waypoint"
18 constexpr double STOP_EPS = 0.02; ~> Si el robot se mueve menos de esto entre ciclos se considera parado. Cuánto puede equivocarse el giro
19 constexpr double SPEED_EPS = 0.05; ~> Si la velocidad estimada es menor que esto, se considera MUY LENTA.
20 ④ // FUNCIONES AUXILIARES (UTILS) ~> Estas funciones hacen tareas pequeñas para no repetir código.
21 // ===== Utils =====
22 double planarDistance(double x1, double y1, double x2, double y2) { DISTANCIA EN EL PLANO XY
23 { - Calcula la distancia entre 2 puntos en 2D.
24     return std::hypot(x1 - x2, y1 - y2); - hypot (dx, dy) es más estable que sqrt (dx*dx + dy*dy)
25 } ~>  $\sqrt{a^2+b^2}$  → Es la distancia típica de un mapa.
26 // Error de yaw normalizado ~> Calcula que tan mal está orientado el robot.
27 double yawError(double yaw1, double yaw2) { Calcular la diferencia angular mínima entre dos ángulos
28 {
29     double err = yaw1 - yaw2; Diferencia directa de "yaw"
30     while (err > M_PI) err -= 2.0 * M_PI; } Normaliza el ángulo al rango  $[-\pi, \pi]$  Sirve para que:
31     while (err < -M_PI) err += 2.0 * M_PI; } 171° y -171° no se consideren "muy lejos"
32     return std::fabs(err); Devuelve el error absoluto (sin signo) } El error está siempre entre  $-\pi$  y  $\pi$ 
33 }
34 // Cuaternión "yaw" ~> los robots utilizan cuaterniones para rotar, los humanos preferimos los ángulos, por ello
35 double quatToYaw(const geometry_msgs::msg::Quaternion &q) ~> Convierte una orientación ROS a "yaw".
36 {
37     tf2::Quaternion tf_q; } - Convierte el mensaje ROS a TF2.
38     tf2::fromMsg(q, tf_q); } - Extrae el ángulo de yaw (rotación en Z). }
39     return tf2::getYaw(tf_q); } Convierte cuaternión → yaw (rotación sobre Z)
40 }
41 ⑤ // ESTRUCTURA Waypoint
42 // ===== Waypoint =====
43 struct Waypoint {
44 {
45     double x; } - Define un waypoint con: posición (x,y)
46     double y; } - orientación "yaw"
47     double yaw; } - Un waypoint es una posición (x,y)
48 }; } ~> un giro "yaw"
49 ⑥ // FUNCIÓN PRINCIPAL
50 int main(int argc, char **argv) ~> Punto de entrada del programa (Aquí empieza el programa)
51 {
52     rclcpp::init(argc, argv); ~> Inicializa ROS ; Arranca ROS
53     auto node = rclcpp::Node::make_shared("patrol_waypoints_tf2_yaw_v2"); ~> Crea el nodo con ese nombre.
54
55     auto tf_buffer = std::make_shared<tf2_ros::Buffer>(node->get_clock()); ~> Almacena transformaciones TF.
56     auto tf_listener = std::make_shared<tf2_ros::TransformListener>(*tf_buffer); ~> Escucha automáticamente las transformaciones.
57
58     auto goal_pub = // PUBLICADOR DE GOALS
59     { node->create_publisher<geometry_msgs::msg::PoseStamped>("/goal_pose", 10); } ~> Este nodo manda órdenes al robot: "Ve a esta pose"
60     ~> Publica objetivos en /goal_pose. Cola de 10 mensajes.
61 ⑦ // LISTA DE WAYPOINTS
62 std::vector<Waypoint> waypoints = {
63     {-3.89, 7.86, -0.0},
64     {-7.32, 7.58, -0.0},
65     {-4.34, 7.22, -0.0},
66     {-4.20, 3.26, -0.0} } RUTA QUE EL ROBOT DEBE PATRULLAR
67 };

```

Zela

```

68 ⑧ // VARIABLES DE ESTADO
69     size_t wp_index = 0; ~ Índice del waypoint actual
70     rclcpp::Time last_goal_sent = node->get_clock()->now(); ~ Marca cuándo se publicó el último "goal"
71
72     double last_x = 0.0; } Posición anterior (para estimar velocidad).
73     double last_y = 0.0;
74
75     bool position_reached = false; ~ Controla la fase 1 / fase 2; Sirve para saber:
76 ⑨ // MENSAJE 'goal' PRECONFIGURADO
77     geometry_msgs::msg::PoseStamped goal_msg; ~ Todos los goals estarán en el frame "map"
78     goal_msg.header.frame_id = "map";
79 ⑩ // Loop PRINCIPAL
80     rclcpp::WallRate loop_rate(10); ~ Loop a 10 Hz
81
82     RCLCPP_INFO(node->get_logger(), "Patrulla iniciada (2 fases: posición + yaw)"); ~ Mensaje informativo
83 ⑪ // BUCLE WHILE ~ "Mientras ROS siga vivo y queden puntos" ~ ROS esté activo
84     while (rclcpp::ok() && wp_index < waypoints.size()) ~ Corre mientras ~ Queden 'waypoints'
85     {
86         rclcpp::spin_some(node); ~ Spin parcial; Procesa callbacks pendientes.
87
88         double rx, ry, ryaw;
89
90         try
91         { // LECTURA DE TF
92             auto tf = tf_buffer->lookupTransform(
93                 "map", "base_link", tf2::TimePointZero); ~ Obtiene la pose actual del robot.
94                                     ~ Pregunta: "¿Dónde está el robot en el mapa?"
95
96             rx = tf.transform.translation.x;
97             ry = tf.transform.translation.y;
98             ryaw = quatToYaw(tf.transform.rotation); ~ Extrae posición y "yaw"
99                                     ~ Si falla:
100         }
101         catch (tf2::TransformException &e) ~ Muestra advertencia, espera y continua.
102         {
103             RCLCPP_WARN_THROTTLE(
104                 node->get_logger(), *node->get_clock(), 2000,
105                 "Esperando TF map->base_link");
106             loop_rate.sleep();
107             continue;
108         }
109         ~ delta-move = distancia desde el último ciclo
110         ~ Si se mueve poco -> está parado
111 ⑫ // ESTIMACIÓN DE MOVIMIENTO
112     double delta_move = planarDistance(last_x, last_y, rx, ry); ~ Estima velocidad usando distancia
113     double est_speed = delta_move * 10.0; ~ y frecuencia (10 Hz).
114     bool stopped = delta_move < STOP_EPS; ~ Detecta si el robot está parado
115 ⑬ // PUBLICACIÓN PERIÓDICA DEL "goal":
116     // ---- Publicar goal periódicamente ----
117     if ((node->get_clock()->now() - last_goal_sent).seconds() > 1.0 || stopped) ~ Publica ~ cada 1s
118     { ~ o si el robot
119         ~ está detenido
120         const auto &wp = waypoints[wp_index];
121
122         goal_msg.header.stamp = node->get_clock()->now();
123         goal_msg.pose.position.x = wp.x;
124         goal_msg.pose.position.y = wp.y;
125
126         tf2::Quaternion q;
127         q.setRPY(0.0, 0.0, wp.yaw); ~ Orientación
128         goal_msg.pose.orientation = tf2::toMsg(q);
129
130         goal_pub->publish(goal_msg); ~ Se publica
131         last_goal_sent = node->get_clock()->now();
132     }
133
134     last_x = rx;
135     last_y = ry;
136
137     const auto &wp = waypoints[wp_index];
138 ⑭ // ERRORES ACTUALES:

```

Zela

```

134 double dist_err = planarDistance(rx, ry, wp.x, wp.y);
135 double yaw_err = yawError(ryaw, wp.yaw);
136 //Tolerancia adaptativa:
137 double dist_tol =
138     (est_speed < SPEED_EPS) ? DIST_TOL_CLOSE : DIST_TOL_FAR;
139 (15) // FASE 1: Llegar a la posición
140 // ----- FASE 1: posición -----
141 if (!position_reached && dist_err < dist_tol) Cuando se alcanzan la posición
142 {
143     position_reached = true;
144     RCLCPP_INFO(node->get_logger(),
145         "WP %zu: posición alcanzada, ajustando orientación...",
146         wp_index + 1);
147 }
148 (16) // FASE 2: ajustar yaw (girar)
149 // ----- FASE 2: yaw -----
150 if (position_reached && yaw_err < YAW_TOL) Cuando posición y orientación están bien
151 {
152     RCLCPP_INFO(node->get_logger(),
153         "Waypoint %zu COMPLETADO (dist=%.2f, yaw=%.2f)",
154         wp_index + 1, dist_err, yaw_err);
155
156     position_reached = false;
157     wp_index++;
158 }
159
160 loop_rate.sleep();
161 }
162 (17) // FINALIZACIÓN
163 RCLCPP_INFO(node->get_logger(), "Ruta finalizada correctamente");
164 rclcpp::shutdown();
165 return 0;
166 }

```

Handwritten notes:

- dist_err = distancia al waypoint
- yaw_err = error de orientación
- Se activa la fase de orientación
- No cambia de waypoint aún.
- Dice: "Ya estoy en el sitio, ahora girale"
- Waypoint completado
- Se avanza al siguiente
- Dice: "Ya estoy bien colocado, siguiente punto"
- Cierra ROS limpiamente

① ¿QUÉ PASA SI LA VELOCIDAD ESTIMADA SE CONSIDERA MUY LENTA?

La velocidad estimada se calcula aquí:

```

double delta_move = planarDistance(last_x, last_y, rx, ry);
double est_speed = delta_move * 10.0;

```

📌 ¿Qué significa esto?

- delta_move → cuánto se movió el robot entre dos iteraciones
- El bucle corre a 10 Hz
- * 10.0 ≈ distancia / segundo → **velocidad estimada**

Luego se compara con:

```

cpp
constexpr double SPEED_EPS = 0.05;

```

[Copiar código](#)

🔍 Caso: velocidad MUY lenta

```

cpp
(est_speed < SPEED_EPS)

```

[Copiar código](#)

Cuando esto ocurre:

```

cpp
double dist_tol =
    (est_speed < SPEED_EPS) ? DIST_TOL_CLOSE : DIST_TOL_FAR;

```

[Copiar código](#)

➡ Se usa DIST_TOL_CLOSE = 0.45

En cambio, si va más rápido:

➡ Se usa DIST_TOL_FAR = 0.30

Zela

🧠 ¿Por qué hacer esto?

Si el robot:

- está casi parado
- está maniobrando
- o el planner oscila cerca del objetivo

entonces exigir una tolerancia **muy pequeña** puede hacer que:

- nunca "considere alcanzado" el waypoint
- se quede corrigiendo infinitamente

👉 **Solución:**

cuando va lento, se **relaja la tolerancia de posición**.

🚦 Resumen del efecto

Velocidad estimada	Tolerancia usada	Resultado
Alta	0.30 m	Más precisión
Muy baja / parada	0.45 m	Evita bloqueos

⚠️ **No cambia la velocidad real del robot, solo la lógica de decisión.**

② ¿POR QUÉ APARECE ESE '10' EN EL "create_publisher"?

```
node->create_publisher<geometry_msgs::msg::PoseStamped>(
    "/goal_pose", 10);
```

Ese `10` es el **tamaño de la cola (queue size)**.

📌 ¿Qué es la cola?

Es el número máximo de mensajes que:

- pueden estar **pendientes de envío**
- si el suscriptor no los procesa a tiempo

Si se llena:

➡ **se descartan los mensajes más antiguos**

🧠 ¿Por qué 10?

Porque:

- el goal se publica cada ~1 segundo
- los nodos de navegación suelen procesarlos rápido
- **no es crítico** perder goals antiguos

Valor	Comportamiento
1	Riesgo de perder mensajes
10	Seguro y estándar
100	Memoria innecesaria
0	❌ No permitido

✍️ ¿Cuándo cambiarlo?

- Si publicas a **frecuencia muy alta** → subir
- Si el mensaje es **crítico** → subir
- Si solo importa el último → incluso `1` puede valer

③ ¿POR QUÉ `loop_rate(10)`? ¿QUÉ ES?

```
rclcpp::WallRate loop_rate(10);
```

Esto define la **frecuencia del bucle principal**:

🕒 **10 Hz = 10 veces por segundo**

🚩 ¿Qué hace exactamente?

Al final del bucle:

```
cpp
loop_rate.sleep();
```

➡ Hace que cada iteración dure ~100 ms

🧠 ¿Por qué 10 Hz?

Porque:

- TF no necesita ultra-alta frecuencia
- el robot no se mueve tan rápido
- es suficiente para navegación

🔢 Relación con la velocidad estimada

Esto es CLAVE 📌

```
cpp
est_speed = delta_move * 10.0;
```

Ese `10.0` viene directamente del `loop_rate(10)`

Si cambias el loop a:

```
cpp
rclcpp::WallRate loop_rate(20);
```

🚦 Comparación de frecuencias

Hz	Periodo	Uso típico
5	200 ms	Robots lentos
10	100 ms	Navegación estándar
20	50 ms	Control fino
50+	<20 ms	Control bajo nivel

👉 TAMBIÉN deberías cambiar:

```
cpp
est_speed = delta_move * 20.0;
```

⚠ Si no:

- la velocidad estimada será incorrecta
- las tolerancias se elegirán mal

🧠 Resumen final

◆ Velocidad lenta

- ✓ Activa una **tolerancia mayor**
- ✓ Evita que el robot se quede bloqueado
- ✓ No cambia el movimiento físico

◆ 10 en el publisher

- ✓ Tamaño de cola
- ✓ Valor seguro y estándar
- ✓ Evita pérdida de mensajes

◆ `loop_rate(10)`

- ✓ Controla la frecuencia del bucle
- ✓ Afecta directamente a la estimación de velocidad
- ✓ 10 Hz es un buen equilibrio

VERSION4: AMCL

```

1  #include "rclcpp/rclcpp.hpp" ~ Permite usar ROS2 (nodos, publicar, suscribirse, logs ...)
2  #include "geometry_msgs/msg/pose_stamped.hpp" ~ Mensaje para decirle al robot: "Ve a esta posición en el mapa"
3  #include "geometry_msgs/msg/pose_with_covariance_stamped.hpp" ~ Mensaje que usa AMCL para decir: "Creo que estoy aquí (más o menos)"
4  #include <vector>
5  #include <array>
6  #include <cmath>
7  #include <chrono>
8
9  using namespace std::chrono_literals;
10  ① PARÁMETROS IMPORTANTES
11  // Parámetros
12  const double GOAL_TOLERANCE = 0.4; ~ Si el robot está a menos de 40cm decimos: "Vale, ya has llegado"
13  const int NAV_START_DELAY = 2; ~ Espera 2 segundos antes de empezar (porque Nav2 y AMCL tardan un poco en arrancar)
14  ② VARIABLES GLOBALES
15  // Variables globales AMCL ~ Memoria del robot
16  double amcl_x = 0.0; ~ Aquí guardamos:
17  double amcl_y = 0.0; ~ o dónde dice AMCL que está el robot
18  bool amcl_recibido = false; ~ o si ya hemos recibido algún mensaje de AMCL
19  ③ CALLBACK DE AMCL (el GPS del robot)
20  // Callback AMCL
21  void amclCallback(const geometry_msgs::msg::PoseWithCovarianceStamped::SharedPtr msg)
22  { ~ Esta función se ejecuta cada vez que AMCL publique algo.
23    amcl_x = msg->pose.pose.position.x; ~ Guardamos la posición del robot
24    amcl_y = msg->pose.pose.position.y;
25    amcl_recibido = true; ~ Marcamos: "Ya sé dónde está el robot"
26  }
27  ④ INICIO DEL PROGRAMA
28  int main(int argc, char **argv)
29  {
30    rclcpp::init(argc, argv); ~ Arranca ROS2 ~ Creamos el nodo (el "cerebro")
31    auto node = rclcpp::Node::make_shared("goal_publisher_v4_amcl");
32    ⑤ PUBLICADOR DE OBJETIVOS
33    auto publisher = node->create_publisher<geometry_msgs::msg::PoseStamped>(
34      "/goal_pose", 10); ~ Esto sirve para decirle al robot: "Ve a este sitio"
35    ⑥ SUSCRIPTOR a AMCL ~ Es una cola, por si se envían varios mensajes rápido.
36    auto amcl_sub = node->create_subscription<
37      geometry_msgs::msg::PoseWithCovarianceStamped>(
38      "/amcl_pose",
39      10,
40      amclCallback); ~ Aquí el robot escucha su propia posición.
41      ~ Cada vez que llega un mensaje se ejecuta "amclCallback"
42    ⑦ LISTA DE OBJETIVOS
43    // Waypoints (x, y, z)
44    std::vector<std::array<double, 3>> waypoints = {
45      {-3.89, 7.86, -0.0},
46      {-7.32, 7.58, -0.0},
47      {-4.34, 7.22, -0.0},
48      {-4.20, 3.26, -0.0} ~ Cada uno es (x,y,z)
49    };
50    ⑧ ESPERAR A QUE Nav2 y AMCL ESTÉN LISTOS
51    // Esperar arranque de Nav2 / AMCL
52    rclcpp::sleep_for(std::chrono::seconds(NAV_START_DELAY));
53    ~ El robot se queda quieto 2 segundos (para no mandar goals demasiado pronto)
54    RCLCPP_INFO(node->get_logger(),
55      "Iniciando versión 4: detección de llegada basada en AMCL");
56    rclcpp::Rate rate(5);

```

Zela

9 BUCLE PRINCIPAL

// --- BUCLE DE WAYPOINTS --- Recorre los waypoints UNO a UNO

for (size_t i = 0; i < waypoints.size() && rclcpp::ok(); i++)

{

auto &wp = waypoints[i];

// Crear objetivo

geometry_msgs::msg::PoseStamped goal; *↗ Creamos el mensaje "ve aquí"*

goal.header.frame_id = "map"; *↗ Las coordenadas están en el mapa*

goal.header.stamp = node->get_clock()->now();

goal.pose.position.x = wp[0]; *de decimos a dónde ir*

goal.pose.position.y = wp[1];

goal.pose.position.z = wp[2];

// Orientación (yaw = 0 rad)

goal.pose.orientation.x = 0.0;

goal.pose.orientation.y = 0.0;

goal.pose.orientation.z = 0.0;

goal.pose.orientation.w = 1.0; *↗ Sin girar (yaw=0)*

// Enviar objetivo

publisher->publish(goal); *↗ Se lo mandamos al robot*

RCLCPP_INFO(node->get_logger(),

"Objetivo %zu enviado → (%.3f, %.3f, %.3f)",

i + 1, wp[0], wp[1], wp[2]);

// --- Esperar llegada usando AMCL ---

amcl_recibido = false; *↗ Esperar a que llegue. Reset: "Aún no se dónde está el robot"*

Mientras no haya llegado...

while (rclcpp::ok())

{ *Procesa mensajes de AMCL*

rclcpp::spin_some(node);

Todavía no llegó AMCL?

if (!amcl_recibido) *↗ Esperamos y no hacemos nada más.*

{

RCLCPP_WARN_THROTTLE(

node->get_logger(),

*node->get_clock(),

2000,

"Esperando pose de AMCL...");

rate.sleep();

continue;

}

Calcular distancia al objetivo

double dx = wp[0] - amcl_x;

double dy = wp[1] - amcl_y;

double distancia = std::hypot(dx, dy); *Distancia del robot al objetivo*

RCLCPP_INFO(node->get_logger(),

"Distancia AMCL al objetivo %zu: %.3f m",

i + 1, distancia);

Está lo bastante cerca???

if (distancia < GOAL_TOLERANCE) *→ Si está a menos de 40cm; "Ok, ya llegaste"*

{

RCLCPP_INFO(node->get_logger(),

"Llegada al objetivo %zu (basado en AMCL)",

i + 1);

break;

}

rate.sleep();

}

Pequeña pausa entre objetivos:

rclcpp::sleep_for(500ms); *↗ Medio segundo para que todo se estabilice*

}

10 FIN DEL PROGRAMA

RCLCPP_INFO(node->get_logger(),

"Patrulla completada (versión 4).");

rclcpp::shutdown();

return 0;

}

Salimos del while

Volvemos al siguiente 'waypoint'

Zela

VERSION 3: ODOM

```
1  #include <chrono>
2  #include <vector>
3
4  #include "rclcpp/rclcpp.hpp"
5  #include "rclcpp_action/rclcpp_action.hpp"
6
7  #include "nav2_msgs/action/navigate_to_pose.hpp"
8  #include "nav_msgs/msg/odometry.hpp"
9
10 using namespace std::chrono_literals;
11 using NavigateToPose = nav2_msgs::action::NavigateToPose;
12
13 class MultiGoalNode : public rclcpp::Node
14 {
15 public:
16     MultiGoalNode() : Node("multi_goal_node"), current_goal_(0)
17     {
18         // ----- ODOM (solo para cumplir V3) -----
19         odom_sub_ = create_subscription<nav_msgs::msg::Odometry>(
20             "/odom", 10,
21             [](nav_msgs::msg::Odometry::SharedPtr){});
22
23         // ----- ACTION CLIENT -----
24         client_ = rclcpp_action::create_client<NavigateToPose>(
25             this, "navigate_to_pose");
26
27         goals_ = {
28             {-3.89, 7.86, -0.0},
29             {-7.32, 7.58, -0.0},
30             {-4.34, 7.22, -0.0},
31             {-4.20, 3.26, -0.0}
32         };
33
34         RCLCPP_INFO(get_logger(), "Esperando servidor NavigateToPose...");
35         client_->wait_for_action_server();
36
37         send_next_goal();
38     }
39
40 private:
41     void send_next_goal()
42     {
43         if (current_goal_ >= goals_.size())
44         {
45             RCLCPP_INFO(get_logger(), "Todas las metas completadas");
46             rclcpp::shutdown();
47             return;
48         }
49
50         NavigateToPose::Goal goal;
51         goal.pose.header.frame_id = "map"; // COORDENADAS CORRECTAS
52         goal.pose.header.stamp = now();
53         goal.pose.pose.position.x = goals_[current_goal_].first;
54         goal.pose.pose.position.y = goals_[current_goal_].second;
55         goal.pose.pose.orientation.w = 1.0;
56
57         RCLCPP_INFO(get_logger(),
58             "Enviando meta %zu: (%.2f, %.2f)",
59             current_goal_ + 1,
60             goal.pose.pose.position.x,
61             goal.pose.pose.position.y);
62     }
```

```

63     auto options = rclcpp_action::Client<NavigateToPose>::SendGoalOptions();
64
65     options.result_callback =
66     [this](const rclcpp_action::ClientGoalHandle<NavigateToPose>::WrappedResult & result)
67     {
68         if (result.code == rclcpp_action::ResultCode::SUCCEEDED)
69         {
70             RCLCPP_INFO(get_logger(),
71                         "Meta %zu alcanzada (ACTION)",
72                         current_goal_ + 1);
73             current_goal_++;
74             rclcpp::sleep_for(1s);
75             send_next_goal();
76         }
77         else
78         {
79             RCLCPP_WARN(get_logger(),
80                         "Meta %zu no alcanzada, código %d",
81                         current_goal_ + 1,
82                         static_cast<int>(result.code));
83         }
84     };
85
86     client_>async_send_goal(goal, options);
87 }
88
89 rclcpp::Subscription<nav_msgs::msg::Odometry>::SharedPtr odom_sub_;
90 rclcpp_action::Client<NavigateToPose>::SharedPtr client_;
91
92 std::vector<std::pair<double, double>> goals_;
93 size_t current_goal_;
94 };
95
96 ✓ int main(int argc, char **argv)
97 {
98     rclcpp::init(argc, argv);
99     rclcpp::spin(std::make_shared<MultiGoalNode>());
100    rclcpp::shutdown();
101    return 0;
102 }

```

Zela